Diabetes Readmission Prediction Using Structured Health Data

CS 549 - Machine Learning, San Diego State University, Spring 2026

Niki Liao, Javi Garcia, Adrian Serrano, Jackson Adams

1. Abstract

This project addresses the challenge of predicting hospital readmission within 30 days for diabetic patients using the Diabetes 130-US Hospitals dataset from the UCI Machine Learning Repository. The dataset contains 101,766 inpatient records collected across 130 U.S. hospitals over 10 years (1999-2008), each with approximately 50 clinical, demographic, and administrative features. We developed a preprocessing pipeline to handle non-standard missing values, high-cardinality ICD-9 codes, duplicate patient encounters, and severe class imbalance. We then trained and compared four classification models: Logistic Regression, Random Forest, K-Nearest Neighbors, and XGBoost. All models were tuned using cross-validation with F1 score as the optimization target, and threshold optimization was applied to improve recall on the minority class. XGBoost achieved the best overall performance with an F1 score of 0.2396 and the highest recall of 0.3936, while Logistic Regression provided a strong baseline with comparable results. Our findings confirm that 30-day readmission prediction from structured hospital records remains a difficult task, consistent with published benchmarks on this dataset.

2. Introduction

2.1 Background and Motivation

Hospital readmission refers to when a patient returns to the hospital within a specified time after discharge. It is one of the most closely tracked quality indicators in healthcare. Unplanned readmissions are costly, with estimates exceeding $25 billion per year in Medicare spending alone [8]. Beyond the financial burden, readmitted patients tend to experience higher mortality rates, longer subsequent stays, and lower overall quality of life compared to those who recover without returning.

In 2012, the Centers for Medicare and Medicaid Services (CMS) introduced the Hospital Readmissions Reduction Program, which penalizes hospitals with excess readmission rates for specific conditions such as heart failure, pneumonia, and acute myocardial infarction. Although diabetes is not currently included in these penalty conditions, diabetic patients are among the most frequently readmitted due to the chronic nature of the disease and the difficulties associated with managing insulin regimens and lifestyle factors [2].

This creates a strong motivation for building predictive models that can identify high-risk diabetic patients before discharge. By flagging these patients, clinicians can intervene with more targeted discharge planning, medication counseling, and follow-up programs. Our goal is to build models that augment clinical

judgment by providing data-driven risk estimates and help allocate limited hospital resources more effectively.

2.2 Dataset Overview

We used the Diabetes 130-US Hospitals dataset, which covers the years 1999 to 2008 and is hosted on the UCI Machine Learning Repository [1]. It is one of the most widely used benchmarks for healthcare readmission prediction research. The dataset contains 101,766 inpatient encounter records, where each patient had a diabetes diagnosis and stayed at least one day in the hospital.

Each record has approximately 50 features spanning several categories. Patient demographics include age, gender, and race. Admission details cover the type of admission (elective, urgent, or emergency), admission source (referral, emergency room, or transfer), and discharge disposition (home, another facility, or expired). Clinical measurements include length of stay, number of lab procedures, number of non-lab procedures, and number of prescribed medications. Diagnosis codes are recorded as ICD-9 strings across three columns representing primary, secondary, and tertiary diagnoses. Additionally, 23 medication columns track whether each drug dosage was increased, decreased, held steady, or not prescribed.

The target variable **readmitted** takes three values: less than 30 days, greater than 30 days, and no readmission. The distribution is heavily skewed, with approximately 11% for less than 30, 35% for greater than 30, and 54% for no readmission [2]. We binarized this into a two-class problem where less than 30 days maps to 1 and everything else maps to 0, resulting in roughly a 9% positive class ratio after preprocessing.

2.3 Dataset Challenges

This dataset is well known for several practical data quality issues that make it realistic and instructive for machine learning [9]. These challenges reflect the messiness of real-world electronic health records.

Non-standard missing value encoding. Instead of using NaN or empty fields, the dataset uses the '?' character for missing values [1]. This affects multiple columns, with the weight column missing roughly 97% of records and payer_code missing about 40%. Different imputation strategies are needed depending on the column.

High-cardinality categorical features. The three diagnosis columns contain ICD-9 codes with hundreds of unique values. Applying one-hot encoding directly would produce an extremely sparse feature matrix. We addressed this by grouping codes into clinically meaningful categories like Circulatory, Respiratory, Diabetes, and others.

Duplicate patient encounters. Some patients appear multiple times in the dataset due to multiple hospital visits. If the same patient appears in both training and test sets, the model could learn patient-specific patterns rather than generalizing. We resolved this by keeping only the first encounter per patient.

Severe class imbalance. Only about 11% of encounters result in readmission within 30 days. A naive model predicting all patients as not readmitted would achieve roughly 89% accuracy but provide no clinical value. This required us to use techniques like SMOTE [6] and threshold optimization, and to focus on metrics like recall and F1 rather than accuracy alone.

Feature redundancy and noise. Some features like encounter_id are identifier codes with no predictive value. Others like admission_type_id are encoded as integers but actually represent unordered categories. Several medication columns have extremely low variance (over 99% of values the same), contributing noise rather than signal.

2.4 Project Objectives

Our project had four main objectives. First, we aimed to develop a robust preprocessing pipeline that systematically addresses the data quality issues described above to produce a clean dataset suitable for modeling. Second, we wanted to train and tune four different classification models spanning different algorithmic families: a linear classifier (Logistic Regression), a bagging ensemble (Random Forest), an instance-based classifier (K-Nearest Neighbors), and a sequential boosting ensemble (XGBoost). Third, we planned to conduct a thorough comparative analysis evaluating all models across multiple metrics with visualizations including ROC curves, confusion matrices, and feature importance plots. Fourth, we aimed to reflect on the clinical implications of our findings and discuss what the results mean for real-world hospital settings.

3. Related Work

Readmission prediction for hospital patients has been a growing focus of machine learning research. Much of the work on the UCI Diabetes 130-US Hospitals dataset builds on Strack et al. [2], who showed that the HbA1c marker contributed to lower readmission rates. Their study also established the preprocessing conventions we followed, including binarizing the readmission target and filtering out non-readmittable patients such as those who were deceased or transferred to hospice.

Beyond this dataset, readmission prediction research commonly uses Logistic Regression as a baseline due to its high interpretability. Ensemble models like Random Forest [4] and XGBoost [5] have consistently outperformed linear models by capturing nonlinear feature interactions. Several studies have also highlighted the persistent challenge of class imbalance in healthcare datasets. Techniques like SMOTE [6] and cost-sensitive learning have shown measurable improvements in minority class recall.

K-Nearest Neighbors has been explored in healthcare classification tasks as a simple non-parametric approach, though it tends to struggle with high-dimensional feature spaces due to the curse of dimensionality [3]. In our work, we included KNN to provide a contrast with the parametric and ensemble models, testing whether a purely distance-based approach could capture useful patterns in this data.

In our project, we build on these findings by comparing four models under a consistent preprocessing pipeline on the same dataset. Our contribution is a unified comparison that includes threshold optimization across all models, which many prior studies do not perform despite the severe class imbalance in this dataset.

4. Methodology

This section covers our data preprocessing pipeline (4.1), model selection and justification (4.2), hyperparameter tuning strategy (4.3), and evaluation plan (4.4). The overall workflow is: (1) load and explore the dataset, (2) clean and preprocess, (3) split into training and test sets, (4) balance the training set with SMOTE, (5) train and tune all four models, and (6) evaluate and compare. All code was implemented in Python using scikit-learn [7], XGBoost [5], pandas, and NumPy in a single Jupyter notebook.

4.1 Data Preprocessing

4.1.1 Initial Data Loading and Exploration

We loaded the dataset using pandas with the na_values parameter set to '?' so that the non-standard missing values were converted to NaN automatically during loading. We then performed exploratory analysis to understand the distributions, data types, and missing value patterns across all features. We examined the target variable distribution to confirm the class imbalance ratios reported in the literature [2].

4.1.2 Handling Missing Values

Our strategy for handling missing values depended on how much data was missing per column and how useful the column was likely to be. The weight column (missing approximately 97% of values) and payer_code column (missing approximately 40%) were dropped entirely, as imputing them would introduce more noise than useful signal. For payer_code specifically, imputing insurance type information carries a risk of introducing socioeconomic bias.

The medical_specialty column had about 49% missing values but was retained with missing values filled as 'Unknown', since the admitting physician's specialty may still carry predictive signal even when partially observed. For columns with smaller amounts of missing data, including race (approximately 2% missing) and the three diagnosis columns (less than 1% missing each), we used mode imputation. A final pass checked for any remaining missing values and applied median imputation for numeric columns and mode imputation for categorical columns.

4.1.3 Removing Duplicate and Redundant Records

Some patients appeared multiple times in the dataset due to multiple hospital admissions over the study period. Including duplicate records in both training and test sets could lead to data leakage, where the model learns patient-specific patterns rather than generalizable risk factors. To prevent this, we sorted records by encounter_id and kept only the first encounter per unique patient_nbr, reducing the dataset from 101,766 to approximately 71,500 records. We also removed encounters where the discharge_disposition_id indicated the patient died or was transferred to hospice care (disposition codes 11, 13, 14, 19, 20, 21), as these patients cannot be readmitted. Finally, we dropped the encounter_id and patient_nbr columns since they are identifier codes with no predictive value.

4.1.4 Feature Encoding

Our dataset required different encoding strategies for different types of features. The three diagnosis columns contained raw ICD-9 codes with hundreds of unique values. One-hot encoding these directly would produce an extremely sparse feature matrix, so we grouped each code into one of approximately 10 clinically meaningful categories based on standard ICD-9 code ranges (for example, codes 390-459 mapped to Circulatory, 250.xx mapped to Diabetes, codes 460-519 mapped to Respiratory). These grouped categories were then one-hot encoded.

The 23 medication columns, each recording whether a drug dosage was changed, were ordinally encoded as No=0, Steady=1, Down=2, Up=3. The age column, already in 10-year brackets, was encoded ordinally from 0 to 9. Gender was binary encoded (Female=0, Male=1), and the A1Cresult and max_glu_serum columns were ordinally encoded based on their clinical ordering. The admission_type_id, discharge_disposition_id, and admission_source_id columns were converted from integers to strings and one-hot encoded since they represent unordered categories. Medication columns where over 99% of values were the same were dropped as they contribute minimal variance. The medical_specialty column was reduced to the top 10 most common specialties plus an 'Other' category before one-hot encoding.

4.1.5 Feature Scaling

Our features have differing value ranges after encoding. Models like Logistic Regression and KNN are sensitive to feature magnitude, so we applied z-score standardization using StandardScaler to transform all features to zero mean and unit variance. The scaler was fit only on the training set and then applied to both training and test sets to prevent information leakage from the test set into the training process. Although Random Forest and XGBoost are tree-based models that do not require scaling, we trained them on scaled data as well to keep the comparison fair and consistent.

4.1.6 Handling Class Imbalance

The positive class (readmission within 30 days) represents only about 9% of encounters after preprocessing. To address this, we used two approaches. First, we applied SMOTE (Synthetic Minority Oversampling Technique) [6] to the training set only, generating synthetic minority class samples by interpolating between existing instances in the feature space. This balanced the training set to a 50-50 class ratio. Second, all models were configured with class_weight='balanced' to adjust the loss function inversely proportional to class frequencies. SMOTE was applied strictly after the train-test split so that no synthetic data influenced the test set evaluation.

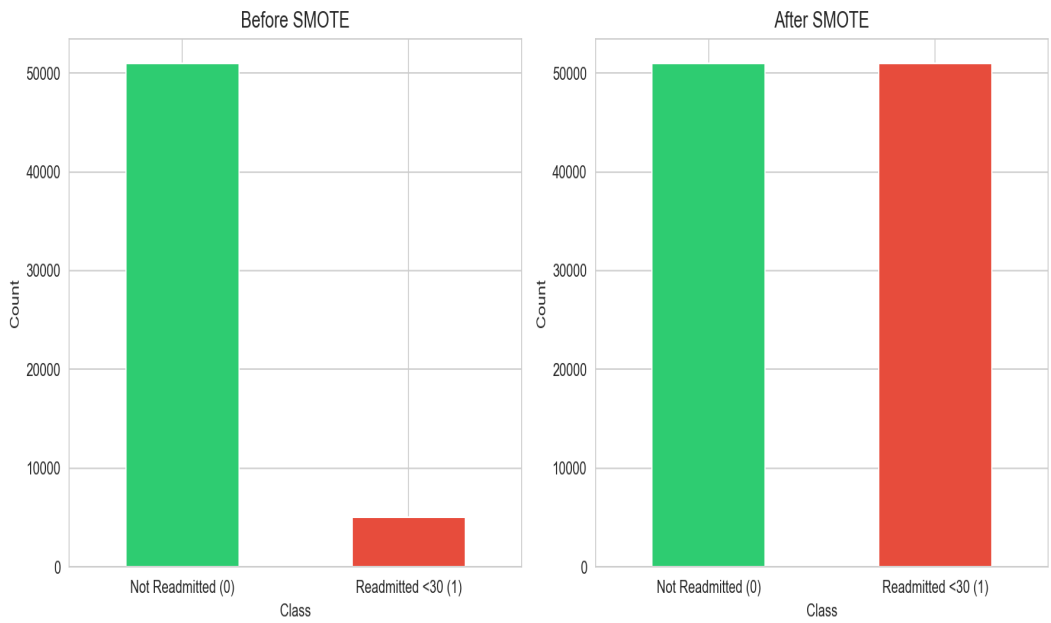

Figure 1. Class distribution before and after applying SMOTE to the training set.

4.1.7 Target Variable Transformation and Train/Test Split

The original target variable has three classes: less than 30 days, greater than 30 days, and no readmission. We binarized this so that less than 30 days maps to 1 and both greater than 30 days and no readmission map to 0. This aligns with our clinical goal of identifying patients at highest risk of early readmission [2]. After preprocessing, we split the data 80/20 using stratified sampling to preserve the class distribution in both sets. The random seed was fixed at 42 for reproducibility.

4.2 Model Selection

4.2.1 Logistic Regression (Adrian)

Logistic Regression serves as our baseline model. It is well suited for binary classification and provides high interpretability since each learned coefficient indicates how much a feature increases or decreases the predicted probability of readmission. For example, a large positive coefficient on number_inpatient means patients with more prior inpatient visits are estimated to have higher readmission risk. This interpretability is important in clinical settings where predictions need to be explainable and trustworthy. Logistic Regression also serves as a reference point: if more complex models only marginally outperform it, that suggests the underlying relationships are largely linear and added complexity may not be worth the cost.

4.2.2 Random Forest (Niki)

Random Forest is a bagging ensemble method that trains multiple decision trees on random subsets of the data and aggregates predictions by majority vote [4]. Its advantages include handling mixed feature types, capturing nonlinear relationships and feature interactions, and resistance to overfitting through the averaging of many independent trees. We used scikit-learn's RandomForestClassifier and tuned hyperparameters including the number of estimators, maximum tree depth, minimum samples for splitting, minimum samples per leaf, and the number of features considered at each split. We used RandomizedSearchCV with 50 iterations since the full parameter grid was too large for exhaustive search.

4.2.3 K-Nearest Neighbors (Javi)

K-Nearest Neighbors (KNN) is an instance-based classifier that assigns a label based on the majority class among the k closest training examples [3]. Unlike the other models, KNN does not learn explicit parameters during training. Instead, it stores the entire training set and performs distance calculations at prediction time. We chose KNN to provide a contrast with the parametric and ensemble models, testing whether a purely distance-based approach could capture useful patterns in this data. We tuned the number of neighbors, the weighting scheme (uniform vs. distance), and the distance metric (Euclidean vs. Manhattan) using GridSearchCV with 5-fold cross-validation.

4.2.4 XGBoost (Jackson)

XGBoost (Extreme Gradient Boosting) is a sequential boosting ensemble where each new tree is trained to correct the residual errors of the previous trees, combining many weak learners into a single strong classifier [5]. Unlike Random Forest which builds trees independently in parallel, XGBoost builds trees sequentially so each iteration addresses the mistakes of the current model. It uses L1 and L2 regularization and branch pruning to prevent overfitting. We tuned learning rate, maximum depth, number of estimators, subsampling ratio, column sampling ratio, minimum child weight, and gamma using RandomizedSearchCV with 50 iterations.

4.3 Hyperparameter Tuning Strategy

Hyperparameter tuning was essential for a fair comparison. Without tuning, differences in model performance could be attributed to poor default settings rather than genuine differences in model capability. All models were tuned using 5-fold stratified cross-validation with F1 score as the optimization target. F1 was chosen because it balances precision and recall, which is important given the class imbalance. Logistic Regression and KNN used GridSearchCV for exhaustive search. Random Forest and XGBoost used RandomizedSearchCV with 50 iterations due to their larger parameter spaces. All tuning was performed on the SMOTE-balanced training set, and final evaluation was done on the unmodified test set.

4.4 Evaluation Metrics

In clinical settings, false negatives (missing a high-risk patient) are much more costly than false positives (flagging a low-risk patient) [8]. This guided our choice of evaluation metrics. We evaluated each model using accuracy, precision, recall, F1 score, ROC AUC, and confusion matrices. Recall was our most important metric since missing a readmission-prone patient has greater clinical consequences than a false alarm. F1 score was our primary comparison metric as it captures the balance between precision and recall.

We also applied threshold optimization to each model. The default 0.5 classification threshold performs poorly on imbalanced data because the model tends to predict the majority class almost exclusively. By sweeping thresholds from 0.1 to 0.9 and selecting the one that maximizes F1, we obtained much more clinically useful predictions at the cost of some overall accuracy. Additional visualizations include ROC curves, precision-recall curves, calibration curves, probability distribution histograms, radar charts, feature importance plots, and runtime comparisons.

5. Results

After preprocessing, the final dataset contained approximately 68,000 records with about 115 features. The training set had roughly 54,000 samples (balanced to about 98,000 after SMOTE) and the test set had approximately 14,000 samples. All four models were trained, tuned, and evaluated on the same data splits.

5.1 Summary Table

Table 1 shows the performance of all four models on the test set using their optimized thresholds.

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC	Threshold	Train Time (s)
Logistic Regression	0.7899	0.1629	0.3243	0.2168	0.6330	0.59	152.1
Random Forest	0.7998	0.1613	0.2932	0.2081	0.6038	0.20	275.5
KNN	0.7345	0.1177	0.3020	0.1694	0.5407	0.17	862.2
XGBoost	0.7760	0.1722	0.3936	0.2396	0.6493	0.12	2780.9

Table 1. Model comparison on the test set with optimized thresholds.

XGBoost achieved the best F1 score (0.2396) and the highest recall (0.3936), meaning it correctly identified about 39% of patients who were actually readmitted within 30 days. Logistic Regression was a close second with an F1 of 0.2168 and recall of 0.3243. Random Forest had the highest raw accuracy (0.7998) but lower recall. KNN performed the weakest across all metrics, likely due to the curse of dimensionality given our 115 features.

5.2 ROC Curves

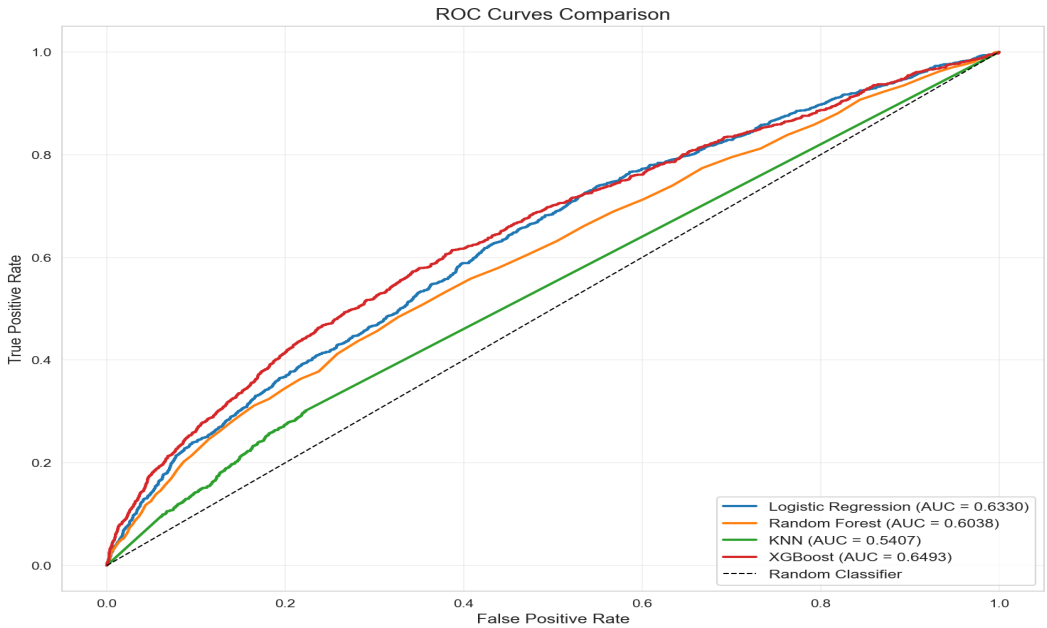

Figure 2. ROC curves for all four models. XGBoost achieved the highest AUC (0.6493), followed by Logistic Regression (0.6330).

The ROC curves show that XGBoost and Logistic Regression provide the best discrimination between classes, with AUC values of 0.6493 and 0.6330 respectively. Random Forest follows at 0.6038, while KNN sits near 0.5407, which is only slightly better than random chance. These AUC values are consistent with

published results on this dataset, where most studies report AUC values in the 0.60 to 0.67 range.

5.3 Precision-Recall Curves

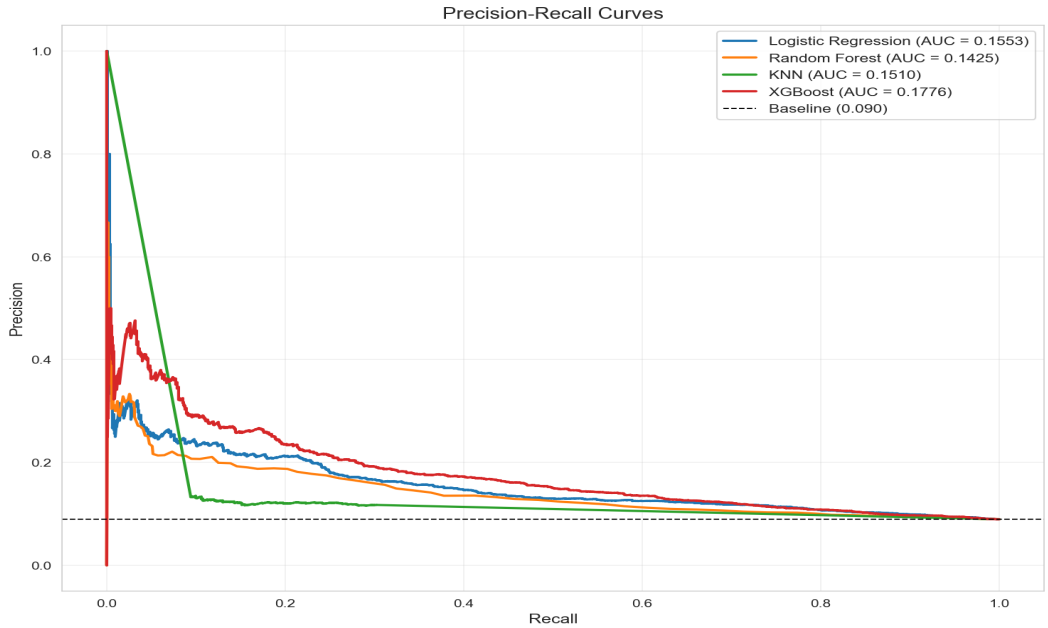

Figure 3. Precision-recall curves. The baseline represents the positive class prevalence in the test set.

The precision-recall curves provide a more informative view than ROC curves for imbalanced datasets. All models show the typical pattern of rapidly declining precision as recall increases, reflecting the difficulty of this prediction task. XGBoost and Logistic Regression maintain slightly higher precision at equivalent recall levels compared to Random Forest and KNN.

5.4 Confusion Matrices

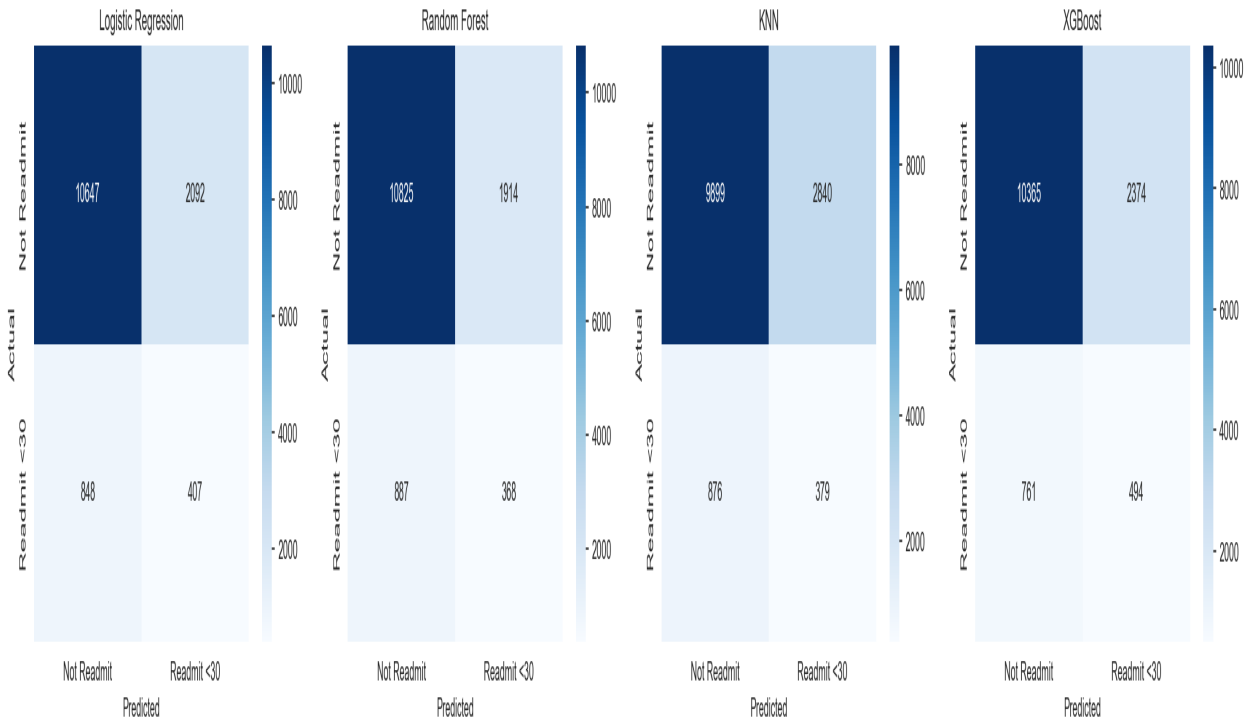

Figure 4. Confusion matrices for all four models using optimized thresholds.

The confusion matrices reveal the tradeoff each model makes between catching readmissions and generating false alarms. XGBoost has the most true positives due to its aggressive threshold (0.12), which also produces a larger number of false positives. Random Forest with its higher threshold (0.20) maintains more true negatives but catches fewer actual readmissions. The pattern across all models is consistent: the true negative cell dominates, and false positives outnumber true positives, reflecting the inherent difficulty of the task.

5.5 Feature Importance

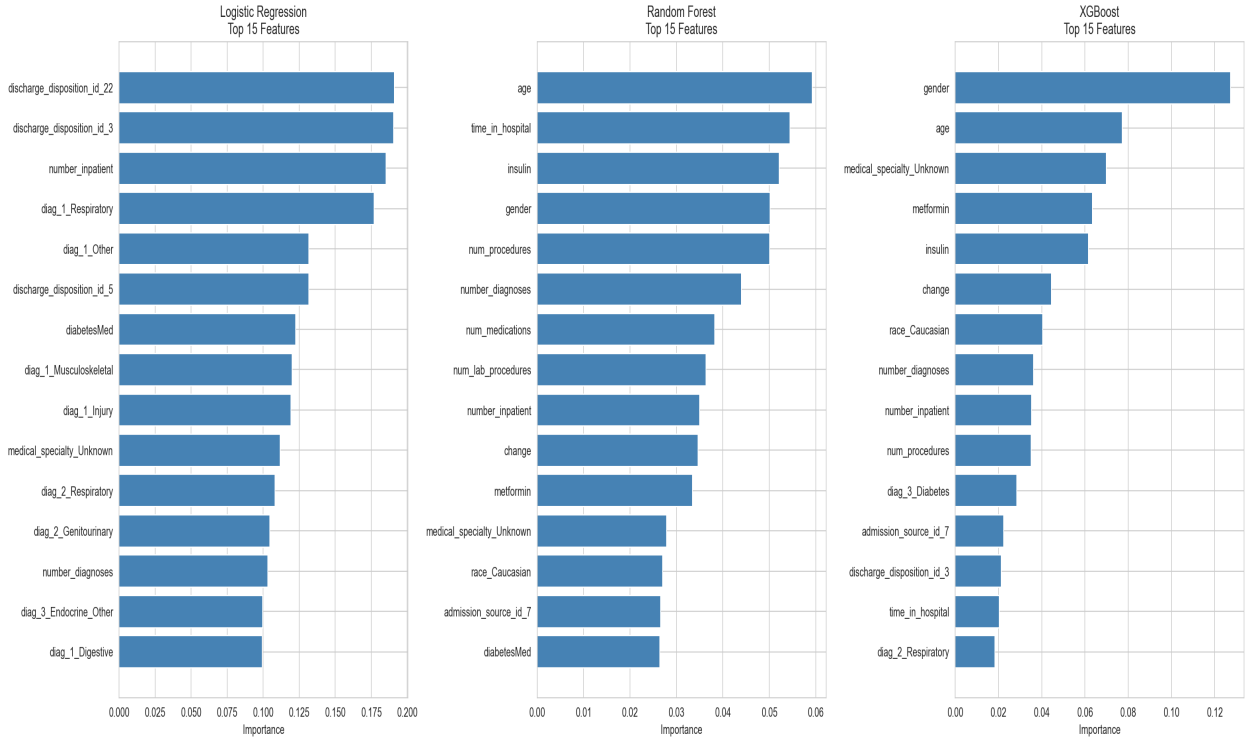

Figure 5. Top 15 most important features for Logistic Regression, Random Forest, and XGBoost.

The feature importance plots reveal some interesting patterns. The number of inpatient visits (number_inpatient) consistently appears as one of the most important features across all three models that provide importance scores. The number of emergency visits and the number of diagnoses also rank highly. Discharge disposition and admission source features appear in the top features for the tree-based models. These results align with clinical intuition, as patients with more prior hospitalizations and more complex medical profiles are at higher risk for readmission. KNN does not provide direct feature importance scores since it relies purely on distance calculations.

5.6 Threshold Analysis

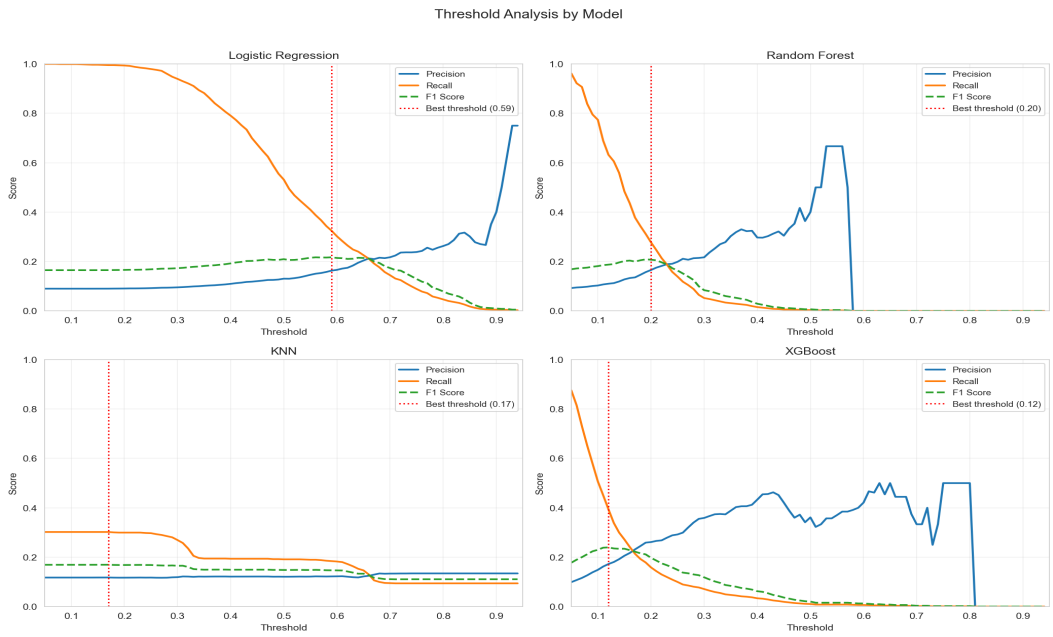

Figure 6. Precision, recall, and F1 as a function of classification threshold. The red line marks the selected threshold.

The threshold analysis plots show how each metric changes as the classification threshold varies. For all models, recall increases as the threshold decreases (more patients are flagged) while precision decreases. The optimal threshold balances this tradeoff by maximizing F1. Notably, the selected thresholds vary significantly across models: Logistic Regression works best at 0.59 while XGBoost peaks at 0.12, reflecting differences in how each model distributes its predicted probabilities.

5.7 Calibration and Probability Distributions

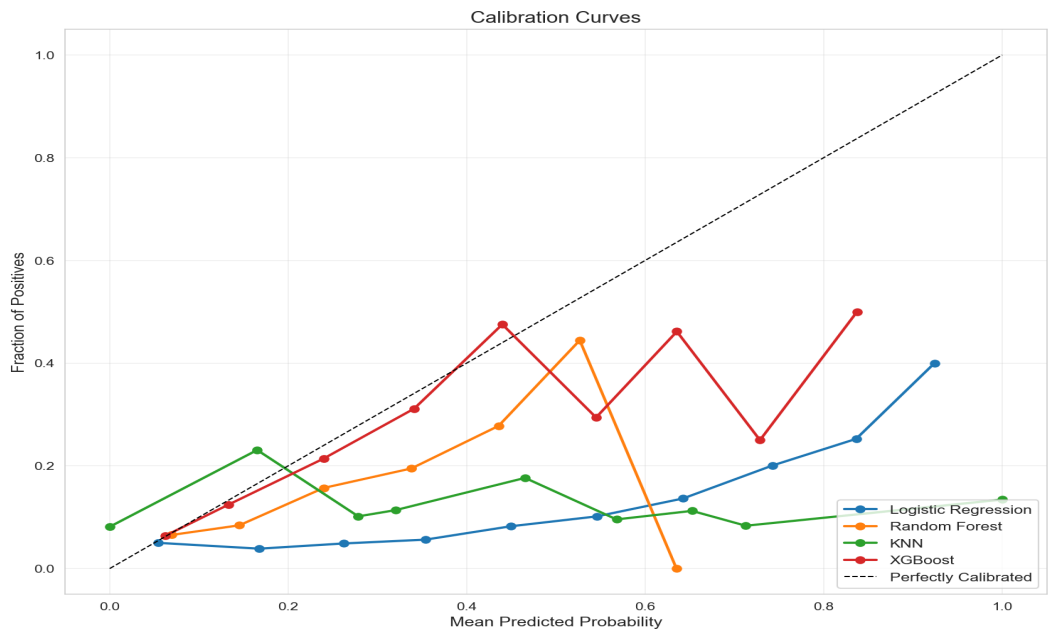

Figure 7. Calibration curves showing how well predicted probabilities match actual outcomes.

The calibration curves reveal that Logistic Regression is the best calibrated model, staying closest to the diagonal of perfect calibration. XGBoost and Random Forest tend to cluster their predictions in certain

probability ranges rather than spreading them evenly. KNN shows the poorest calibration, consistent with its weaker overall performance.

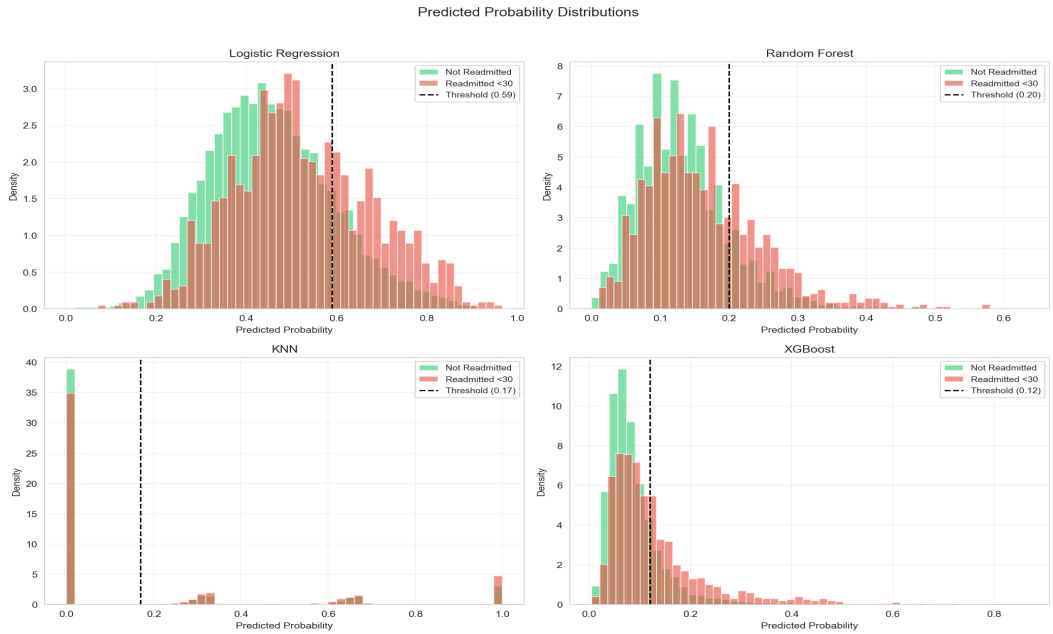

Figure 8. Distribution of predicted probabilities for each class, separated by actual outcome.

5.8 Runtime Comparison

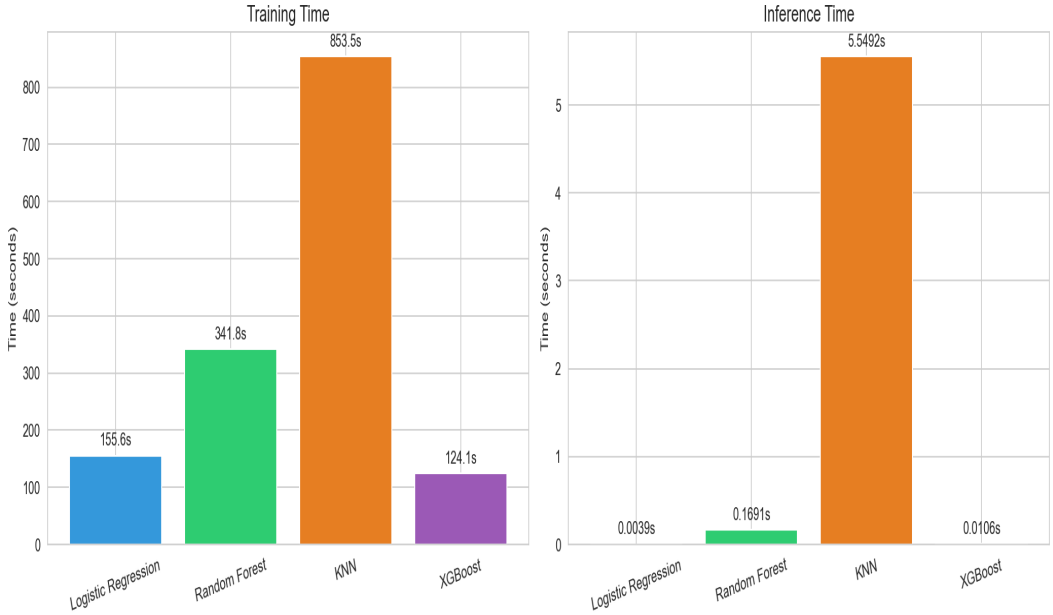

Figure 9. Training and inference time comparison across all models.

XGBoost had the longest training time at approximately 2,781 seconds due to the sequential nature of boosting and the large parameter search. KNN had the second longest at 862 seconds, which is notable since KNN is conceptually simple but computationally expensive at prediction time because it must compute distances to all training examples. Random Forest took 276 seconds and Logistic Regression was the fastest at 152 seconds. For inference time, all models were fast enough for practical use, though KNN was the slowest due to its lazy evaluation approach.

5.9 Radar Chart

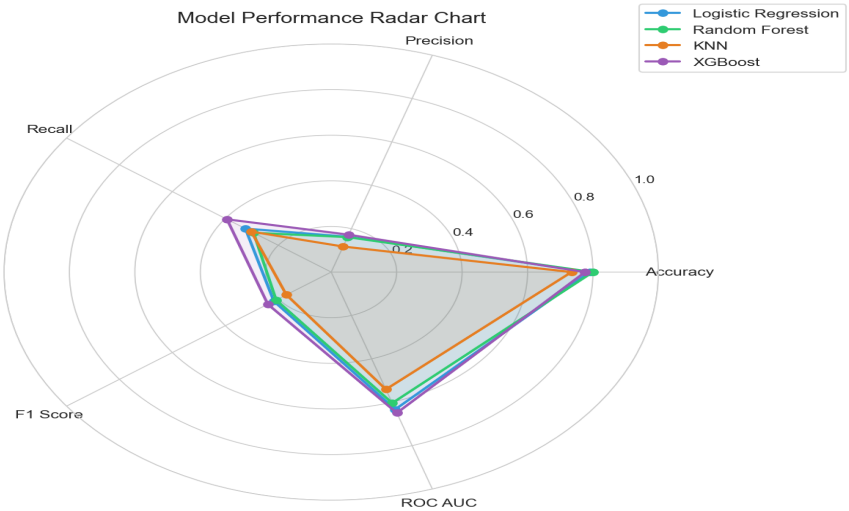

Figure 10. Radar chart showing all five metrics for each model at a glance.

6. Discussion

6.1 Key Findings

Our results confirm that predicting 30-day readmission for diabetic patients from structured hospital records is a genuinely difficult task. The best performing model (XGBoost) achieved an F1 score of 0.2396 and a recall of 0.3936, meaning it correctly identified about 39% of patients who would actually be readmitted. While this may seem low, it is consistent with published benchmarks on this dataset, where AUC values typically fall in the 0.60-0.67 range [2, 9].

One of the most important findings is how misleading accuracy can be for imbalanced data. Random Forest achieved the highest accuracy at 79.98%, but it actually had lower recall than XGBoost (0.2932 vs. 0.3936). Without threshold optimization, all models initially predicted the majority class almost exclusively, producing roughly 91% accuracy but zero recall. This demonstrates that accuracy alone is an inadequate metric for clinical prediction tasks with imbalanced classes.

6.2 Model Comparison

XGBoost emerged as the best overall model with the highest F1, recall, and ROC AUC. Its sequential boosting approach appears well suited to learning subtle patterns in this data. However, it came at a significant computational cost, with training time nearly 20 times longer than Logistic Regression.

Logistic Regression provided a surprisingly competitive baseline, achieving the second-best F1 and AUC scores. The relatively small gap between Logistic Regression and the nonlinear models suggests that much of the predictable signal in this dataset follows roughly linear relationships. This is a meaningful finding because Logistic Regression is far more interpretable and faster to train, making it arguably the better choice for a real clinical deployment where explainability matters.

Random Forest had the highest raw accuracy but lower recall and F1 than both XGBoost and Logistic Regression. Its strength in accuracy reflects a more conservative prediction pattern, catching fewer true positives but also producing fewer false positives.

KNN performed the weakest across all metrics. This was expected given the 115 features in our dataset. KNN relies on distance calculations in the feature space, and in high-dimensional spaces, distances between all points tend to become similar, reducing the algorithm's ability to discriminate [3]. KNN also had the longest inference time because it must compute distances to all training examples for each prediction.

6.3 Bagging vs. Boosting

Comparing Random Forest (bagging) with XGBoost (boosting) provides insight into which ensemble strategy works better for this problem. XGBoost outperformed Random Forest on F1 (0.2396 vs. 0.2081), recall (0.3936 vs. 0.2932), and AUC (0.6493 vs. 0.6038). The sequential error-correction in boosting appears to be more effective at learning the subtle patterns that distinguish readmitted from non-readmitted patients. Random Forest, which builds trees independently, may average out these weak signals across its ensemble.

6.4 Clinical Implications

In a hospital setting, our best model (XGBoost) would flag about 39% of patients who would actually be readmitted within 30 days. While this is far from perfect, identifying even a portion of high-risk patients before discharge can enable targeted interventions such as extra follow-up appointments, medication

review, or post-discharge phone calls. The tradeoff is that the model also generates a significant number of false positives, meaning some resources would be spent on patients who would not have been readmitted. In practice, a hospital would need to tune the threshold based on their available resources and the cost of intervention versus the cost of readmission.

6.5 Limitations

Our project has several limitations. First, the dataset is from 1999-2008, and healthcare practices have evolved since then. Second, many important predictors of readmission, such as patient medication adherence, social support, socioeconomic status, and mental health, are not captured in this data. Third, SMOTE creates synthetic samples by interpolating between existing instances, which may produce unrealistic feature combinations. Fourth, we used a single train-test split rather than repeated cross-validation for the final evaluation, which means our results have some variance. Finally, all models were run on the same machine, but training times may vary depending on hardware and system load, so the runtime comparison should be interpreted directionally rather than as exact benchmarks.

7. Conclusion

In this project, we built a complete machine learning pipeline for predicting 30-day hospital readmission for diabetic patients using the UCI Diabetes 130-US Hospitals dataset. We developed a preprocessing pipeline that addressed missing values, duplicate encounters, high-cardinality features, and severe class imbalance. We trained and compared four classification models: Logistic Regression, Random Forest, K-Nearest Neighbors, and XGBoost.

XGBoost achieved the best overall performance with the highest F1 score (0.2396), recall (0.3936), and ROC AUC (0.6493). However, Logistic Regression was a surprisingly strong baseline, suggesting that much of the signal in this data is approximately linear. KNN struggled with the high-dimensional feature space, confirming the expected limitations of distance-based methods in this setting.

The most important takeaway is that accuracy is misleading for imbalanced classification problems. A model predicting all patients as not readmitted achieves 89% accuracy but provides no clinical value. Threshold optimization was essential to obtain meaningful predictions from all four models.

For future work, we would suggest exploring feature engineering beyond what the raw dataset provides, such as interaction features between medications and diagnoses. Deep learning approaches like neural networks with entity embeddings for categorical features could also be explored. Additionally, incorporating external data sources such as socioeconomic indicators or post-discharge follow-up records could significantly improve prediction performance. Finally, techniques like stacking or blending multiple models could potentially improve upon any individual model's performance.

8. Individual Contributions

Member	Preprocessing	Model	Tuning	Evaluation	Report
Niki Liao	Duplicate removal, feature scaling, train/test split	Random Forest	RandomizedSearchCV with n_estimators, max_depth, etc.	Feature importance and ROC curves	Abstract, Intro, Discussion, Conclusion
Javi Garcia	Feature encoding, ICD-9 grouping, one-hot and ordinal	K-Nearest Neighbors	GridSearchCV with k, weights, metric	KNN analysis and comparison	Methodology KNN and RF

Adrian Serrano	Missing value handling, target binarization	Logistic Regression	GridSearchCV with C, penalty, solver	Confusion matrices, precision-recall curves	Methodology XGBoost and Tuning sections
Jackson Adams	Low variance removal, EDA summary stats	XGBoost	RandomizedSearchCV with lr, depth, etc.	Gain-based importance, bagging vs boosting	Related Work and Results formatting

Table 2. Individual contributions of each team member.

All four team members contributed to both code and report writing. We held weekly team meetings and communicated through a Discord server to coordinate our work. The preprocessing pipeline was a shared effort, with each member taking ownership of specific steps as shown in the table above. Each member implemented, tuned, and evaluated their assigned model. The final comparison, discussion, and report editing were collaborative efforts across the entire team.

9. References

[1] UCI Machine Learning Repository. Diabetes 130-US Hospitals for Years 1999-2008 Dataset. https://archive.ics.uci.edu/dataset/296/diabetes+130-us+hospitals+for+years+1999-2008

[2] Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., & Clore, J. N. (2014). Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records. BioMed Research International, 2014, 781670.

[3] Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21-27.

[4] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5-32.

[5] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 785-794).

[6] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321-357.

[7] Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[8] Jencks, S. F., Williams, M. V., & Coleman, E. A. (2009). Rehospitalizations among patients in the Medicare fee-for-service program. New England Journal of Medicine, 360(14), 1418-1428.

[9] Goldstein, B. A., Navar, A. M., Pencina, M. J., & Ioannidis, J. P. A. (2017). Opportunities and challenges in developing risk prediction models with electronic health records data. Journal of the American Medical Informatics Association, 24(1), 198-208.

[10] Fawcett, T. (2006). An introduction to ROC analysis. Pattern Recognition Letters, 27(8), 861-874.